

Practical-8

```
himasree@LAPTOP-41COT5AA:~$ cd ~/OSLab
himasree@LAPTOP-41COT5AA:~/OSLab$ nano memory.c
himasree@LAPTOP-41COT5AA:~/OSLab$ nano memory1.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc -Wall -g memory1.c -o memory1
himasree@LAPTOP-41COT5AA:~/OSLab$ ./memory1
=== Dynamic Memory Allocation Demo ===

malloc(): 10 20 30 40 50
calloc(): 0 0 0 0 0 (initialized to zero)
realloc(): 10 20 30 40 50 60 70 80 90 100

Memory successfully freed.
himasree@LAPTOP-41COT5AA:~/OSLab$ valgrind --leak-check=full ./memory1
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
himasree@LAPTOP-41COT5AA:~/OSLab$ sudo apt update
sudo apt install valgrind
[sudo] password for himasree:
Sorry, try again.
[sudo] password for himasree:
Sorry, try again.
[sudo] password for himasree:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ valgrind --leak-check=full ./memory1
==1727== Memcheck, a memory error detector
==1727== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1727== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1727== Command: ./memory1
==1727==
=== Dynamic Memory Allocation Demo ===

malloc(): 10 20 30 40 50
calloc(): 0 0 0 0 0 (initialized to zero)
realloc(): 10 20 30 40 50 60 70 80 90 100

Memory successfully freed.
==1727==
==1727== HEAP SUMMARY:
==1727==   in use at exit: 0 bytes in 0 blocks
==1727==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1727==
==1727== All heap blocks were freed -- no leaks are possible
==1727==
==1727== For lists of detected and suppressed errors, rerun with: -s
==1727== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano cow.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc -Wall -g cow.c -o cow
himasree@LAPTOP-41COT5AA:~/OSLab$ ./cow
Parent PID: 1735
Memory allocated: 4000000 bytes

Before fork():
data[0] = 0
data[999999] = 999999

--- Parent Process ---
Parent PID: 1735

--- Child Process ---
Child PID: 1736
Before modification:
Child data[0] = 0

Child modifying data...
After modification:
Child data[0] = 9999

After child modification:
Parent data[0] = 0
himasree@LAPTOP-41COT5AA:~/OSLab$ nano cow.c
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc -Wall -g cow.c -o cow
himasree@LAPTOP-41COT5AA:~/OSLab$ ./cow
Parent PID: 1746
Memory allocated: 4000000 bytes

Parent memory BEFORE fork:
VmSize:      6600 kB
VmRSS:       5644 kB

--- PARENT PROCESS ---
Parent PID: 1746

Parent memory AFTER fork:

--- CHILD PROCESS ---
Child PID: 1749

Child memory BEFORE modification:
VmSize:      6600 kB
VmRSS:       5644 kB
VmSize:      6600 kB
VmRSS:       4972 kB

Child modifying data...

Child memory AFTER modification:
VmSize:      6600 kB
VmRSS:       4972 kB
Child data[0] = 9999
```

```
After child modification:
Parent data[0] = 0

Parent memory AFTER child modification:
VmSize:      6600 kB
VmRSS:       5644 kB
himasree@LAPTOP-41COT5AA:~/OSLab$ |
```

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("=== Dynamic Memory Allocation Demo ===\n\n");

    // 1. malloc()
    int *arr1 = (int *)malloc(5 * sizeof(int));

    if (arr1 == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr1[i] = (i + 1) * 10;
    }

    printf("malloc(): ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr1[i]);
    }
    printf("\n");

    // 2. calloc()
    int *arr2 = (int *)calloc(5, sizeof(int));

```

```

    // 2. calloc()
    int *arr2 = (int *)calloc(5, sizeof(int));

    if (arr2 == NULL) {
        printf("calloc failed\n");
        free(arr1);
        return 1;
    }

    printf("calloc(): ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr2[i]);
    }
    printf("(initialized to zero)\n");

    // 3. realloc()
    arr1 = (int *)realloc(arr1, 10 * sizeof(int));

    if (arr1 == NULL) {
        printf("realloc failed\n");
        free(arr2);
        return 1;
    }

```

```

if (arr1 == NULL) {
    printf("realloc failed\n");
    free(arr2);
    return 1;
}

for (int i = 5; i < 10; i++) {
    arr1[i] = (i + 1) * 10;
}

printf("realloc(): ");
for (int i = 0; i < 10; i++) {
    printf("%d ", arr1[i]);
}
printf("\n");

// 4. free()
free(arr1);
free(arr2);

printf("\nMemory successfully freed.\n");

return 0;

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

void show_memory(const char *message) {
    char command[100];

    printf("\n%s\n", message);

    snprintf(command, sizeof(command),
             "grep -E 'VmRSS|VmSize' /proc/%d/status",
             getpid());

    system(command);
}

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

```

```

    printf("Child data[0] = %d\n", data[0]);

    free(data);
    exit(0);

} else {

    printf("\n--- PARENT PROCESS ---\n");
    printf("Parent PID: %d\n", getpid());

    show_memory("Parent memory AFTER fork:");

    wait(NULL);

    printf("\nAfter child modification:\n");
    printf("Parent data[0] = %d\n", data[0]);

    show_memory("Parent memory AFTER child modification:");

    free(data);
}

return 0;
}

```